Relations in Your Tests

From idea to proposal to plan to working, tested code

COMSC 2043 • Discrete Structures and Critical Thinking

Week 11, October 27, 2026

How to read this deck

PRINCIPLE	A lesson idea this slide demonstrates. Same orange box, top right, every time.
PROMPT FOR CLAUDE	Exact text to paste into Claude. Red box. The file name is in the box corner.
WHAT YOU SHOULD SEE	The output to expect. If it does not match, use the **Recovery** line.
1 PREDICT → **2 RUN** → **3 EXPLAIN**	The same three beats at every step: **predict**, **run**, **explain**.

Parts: **1** the idea • **2** idea to proposal • **3** proposal to pieces and plan • **4** build it live.

Why would anyone write a test first?

A smoke alarm does not fix fires. It tells you, fast and loudly, that something is wrong, *before* the damage is big.

A taste test while cooking: you taste the sauce *before* serving 30 people.

A test is a question with a known answer, written down so a computer can ask it every time.

Test first means: decide what "right" looks like *before* writing the code. Then you know when you are done.

Ask the room (60 seconds)

When have you fixed something, and only afterwards found out it was still broken?

The loop we repeat at every step



1. **PREDICT** before you run: "Will this be red or green? Which test will fail, and why?" Say it out loud, pairs of two.
2. **RUN** the command (`pytest`). Everyone looks at the same screen.
3. **EXPLAIN** what happened in one sentence: "It failed because ..." or "It passed because ..."

One new idea per step. If the screen surprises you, that is information, not failure.

Red tests are normal. Red tests are good.

RED means "the test caught something" $\Rightarrow$ **GREEN** means "now it is true"

- In test-first work, **every step starts red on purpose**. A red screen means the test is working.
- A test that has *never* been red might not be testing anything.
- Feeling lost when the screen is full of red? Everyone does. Read the *last* line first.

Glossary 1 of 3: testing words

test	a question with a known answer, written so a computer can check it.
unit test	a test for one small piece of code, a single function.
pytest	the tool that runs our tests and shows the results.
assert	the line that says "this must be true"; if not, the test fails.
red / green	red: a test is failing. green: all tests pass.
test-driven development (TDD)	write a failing test first, then just enough code to make it pass.
fixture	fixed sample data a test uses, such as a list of names.
stub	an empty placeholder function so a test can run and fail.
partition (bin)	a group of inputs the code treats the same way.
boundary	the edge between two bins, such as 59 and 60.

Glossary 2 of 3: relation words

relation	a set of ordered pairs: which things are linked to which.
ordered pair	(a, b) where order matters: $(2, 6) \neq (6, 2)$.
reflexive	every item is linked to itself.
symmetric	every link works both ways.
antisymmetric	a two-way link only ever joins an item to itself.
transitive	a chain $a \to b \to c$ always has the shortcut $a \to c$.
equivalence relation	reflexive, symmetric and transitive: it sorts things into groups.
partial order	reflexive, antisymmetric and transitive: "comes before" with ties allowed.
property test	a test that checks one of these four rules over many items.
witness	the concrete pair or triple that proves a "no": a counterexample.
canonical key	one standard spelling every variant is rewritten to, then compared.
dependency / cycle	a function that needs another; a loop of such needs.

Glossary 3 of 3: words in the steps

counterexample	one concrete case that shows a rule is false; same idea as a witness.
matrix	a grid of 0s and 1s: row linked to column, 1 means yes.
digraph	dots joined by arrows; one arrow per ordered pair.
edit distance	how many single-letter inserts, deletes or changes turn one word into another.
normalize	rewrite many spellings into one standard form before comparing.
union-find	a way to group items: each match merges two groups. Also called connected components.
leaf	a function that needs no other function; test it first.
property	a rule that must hold for every item, such as symmetric.
PRINCIPLE tag	the orange box saying which lesson idea a slide shows.

Start with a seating chart,
not with notation.

Five students in a row: **Ana, Ben, Cy, Dee, Eli**.
Question: *who sits next to whom?*

A relation is a set of pairs

PRINCIPLE: WITNESS / COUNTEREXAMPLE

Sits next to as a list of pairs (first person, second person):

(Ana, Ben), (Ben, Ana), (Ben, Cy), (Cy, Ben), (Cy, Dee), (Dee, Cy), (Dee, Eli), (Eli, Dee)

- That list *is* the relation: 8 ordered pairs. The word is just a label for "a set of pairs."
- Order matters: (Ana, Ben) and (Ben, Ana) are two different pairs.
- The same relation can be shown as a **matrix** or a **digraph** (arrows).

What you should see: 8 pairs, a 5×5 grid of 0/1, and lines like `Ana -> Ben`. See the prompt on the next slide.

See it: pairs, matrix, digraph

PRINCIPLE: WITNESS / COUNTEREXAMPLE

PROMPT FOR CLAUDE

prompts/01_seating_relation.md

```
Create demos/seating.py. Students: Ana, Ben, Cy, Dee, Eli (in that order, a list named ITEMS).
Relation "sits next to": Ana-Ben, Ben-Cy, Cy-Dee, Dee-Eli, and each pair works both ways.
Store it as a set of ordered pairs named SITS_NEXT_TO (so 8 pairs).
When run as a script, print:
 1. the pairs, sorted;
 2. a 5x5 matrix of 0/1 with the names as row and column labels;
 3. a digraph as text, one line per student listing all neighbours, like "Ben -> Ana, Cy".
Then run: python3 demos/seating.py   and show me the output.
```

What you should see: 8 pairs; a matrix with ones next to the diagonal and zeros on the diagonal; lines `Ana -> Ben`, `Ben -> Ana, Cy`, ..., `Eli -> Dee`.

Recovery: `cp reference/demos/seating.py demos/` then `python demos/seating.py`

1 PREDICT (30 s): How many ones will the matrix have? Which cells on the diagonal?

Four questions about any relation

Reflexive	Is everyone linked to themselves?	Sits next to: **no** (Ana is not next to Ana)
Symmetric	Does every link go both ways?	Sits next to: **yes**
Antisymmetric	Can two *different* people be linked both ways? (must be no)	Sits next to: **no**, Ana and Ben are
Transitive	Does Ana→Ben and Ben→Cy mean Ana→Cy?	Sits next to: **no**, Ana is not next to Cy

- To say **yes**: check every case. To say **no**: show *one* case that breaks it.

Four questions, four witnesses

PROMPT FOR CLAUDE

prompts/02_property_checker.md

```
Create demos/properties.py. Four functions taking (items, pairs): reflexive, symmetric,
antisymmetric, transitive. Each returns None when the property holds, otherwise a witness:
the first failing item, pair, or triple, trying items in the order given.
Add report(title, items, pairs) printing a heading and one line per property:
"holds" or "FAILS  witness: ...".
At the bottom (under a main guard) call report for:
 (a) "sits next to": ITEMS and SITS_NEXT_TO imported from seating.py in the same folder (from seating
   import ...);
 (b) "divides" on the numbers 1..12 (a divides b).
Run python3 demos/properties.py and show me the output.
```

What you should see: sits next to: reflexive FAILS (Ana), symmetric holds, antisymmetric FAILS (Ana, Ben), transitive FAILS (Ana, Ben, Ana). Divides on 1..12: reflexive holds, symmetric FAILS (1, 2), antisymmetric holds, transitive holds.

Recovery: `cp reference/demos/properties.py demos/`

1 PREDICT (30 s): For "divides", which property do you expect to fail, and which pair breaks it?

Two big kinds of relation

Equivalence relation

Reflexive + symmetric + transitive.
"Is the same as." It sorts things into groups.
Same birthday month.

Partial order

Reflexive + antisymmetric + transitive.
"Comes at or before." Some pairs are not comparable.
Divides. Is a prerequisite of.

Our seating relation is **neither**: it fails reflexive and transitive.

Sort relations into kinds

PROMPT FOR CLAUDE

prompts/03_classify_relations.md

```
Create demos/classify.py using the four checkers in demos/properties.py. Print a table with
columns: relation, reflexive, symmetric, antisymmetric, transitive, verdict (yes/no per property).
verdict = "equivalence" if reflexive+symmetric+transitive; "partial order" if
reflexive+antisymmetric+transitive; otherwise "neither".
Relations: (1) same birthday month, on 6 invented students given as (name, month) pairs;
(2) "a divides b" on 1..12; (3) "a <= b" on 1..6; (4) "sits next to" from demos/seating.py;
(5) "a < b" on 1..6. Run it and show me the table.
```

What you should see: verdicts in order: equivalence, partial order, partial order, neither, neither. Row 5 ($<$) fails reflexive, so "neither."

Recovery: `cp reference/demos/classify.py demos/`

1 PREDICT (30 s): Is "less than" ($<$) a partial order? Why or why not?

Where this shows up in real software

Equivalence: deduplicating.
Contacts "Jon", "John", "Jonathan Smith". Same person? Group them, keep one. Hashing and "normalize then compare" rely on this being a true equivalence.

Partial order: "needs first."
Course prerequisites, build order, which function to test first. If two things need each other there is no valid order: a bug in the plan.

If "same person" is not transitive, a duplicate-record tool gives **different answers depending on the order of the records**.

Part 2: From an idea to a written proposal

Before any code: write down
what we are building and how we will know it works.

1. **Problem** in one sentence, in everyday words.
2. **Who it is for.**
3. **Behaviors** as plain sentences ("when X, then Y").
4. **Risks**: what could go wrong.
5. **Done** means: the checks we will run.

A filled-in proposal: duplicate-record merger

Problem The same person appears several times in a list (Jon, John, Jonathan Smith). We want one merged record per person.

For An office assistant who cleans a class list. Synthetic names only today.

Behaviors (1) A score gets exactly one letter grade. (2) Close spellings of a first name can be the same person. (3) "Same person" is a true sameness: if A is B and B is C, then A is C. (4) A match gives the same answer from either side. (5) Sorting never leaves two different people tied. (6) Merging keeps every record exactly once, in any input order. (7) We know which piece to test first.

Risks Merging two real people; missing a true duplicate; nicknames we do not know; chains joining strangers; our own functions needing each other.

Done Every behavior has a test; every property has a witness test; the sample list merges to the expected four lines.

The proposal becomes the test list

Behavior sentence	Becomes
A score gets exactly one letter grade	bins + boundary tests
Close first names can be the same person	a match function + tests
Same person is a true sameness	transitivity property test
A match is the same from either side	symmetry property test
Sorting never ties two different people	antisymmetry property test
Merge keeps every record once	end-to-end test
Know which piece to test first	dependency order

Part 3: Break it into pieces, then order them

Small pieces, each with a test plan,
built from the bottom up.

1. Pull one piece per behavior sentence.
2. Draw what each piece **depends on**.
3. Test the pieces that depend on nothing **first** (the leaves).
4. For each piece: its **bins**, its **boundaries**, its **properties**.

Rules of the game

PROMPT FOR CLAUDE

prompts/00_session_rules.md

```
We are building a small Python project live for a class, using test-driven development.
Rules for the whole session:
- Python 3.9+, standard library plus pytest only.
- Work in the current folder. Package merger/ holds code, tests/ holds tests, demos/ holds
  stand-alone demo scripts. Create a pytest.ini containing: [pytest] / pythonpath = . /
  testpaths = tests   so that "pytest" works from this folder.
- Each time I paste a prompt, do exactly that step and nothing more.
- For test-first steps: write the test first, run pytest, show me the output, and stop.
  Do not write the code that makes it pass until I ask.
- Never write code for a later step. Keep every file short and readable.
Reply "ready" and wait.
```

What you should see: Claude replies "ready" (and may create `pytest.ini`).

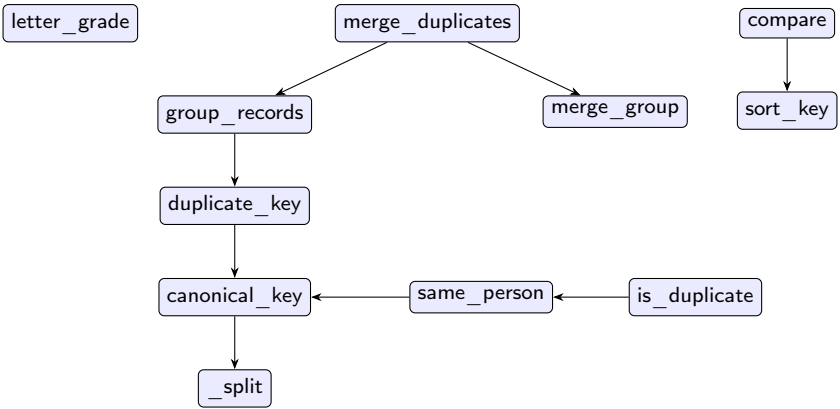
Recovery: none; if Claude drifts later, paste this block again.

1 PREDICT (30 s): Why do we tell Claude not to write the code yet?

The pieces

#	Piece	What it promises
1	`letter_grade`	one letter per score; bins and boundaries
2	`similar_names`	first-draft "looks alike" match
3	`same_person`	a true equivalence (fix for the draft)
4	`is_duplicate`	symmetric from both sides
5	`compare` / `sort_key`	antisymmetric order
6	`merge_duplicates`	groups, merges, loses nothing
7	dependency order	which piece to test first
8	cycle check	refuses an order that cannot exist

Depends on: the plan as arrows



An arrow means "needs." Nothing points back up, so a valid order exists: **test the bottom first**.

Test order: leaves first

1. `letter_grade` **(nothing needed)**: warm-up for bins.
2. `_split`, `edit_distance`, `similar_names`: the first match rule.
3. `same_person`, `canonical_key`: the true equivalence.
4. `is_duplicate`: needs same_person.
5. `sort_key`, `compare`: independent of names.
6. `duplicate_key`, `group_records`, `merge_group`.
7. `merge_duplicates`: needs everything above, so last.

Several orders are valid. Any order where nothing is tested before what it needs works: that is what "partial" means.

Piece 1: letter_grade

PRINCIPLE: EQUIVALENCE PARTITIONING, BOUNDARIES

Bin	Scores	Letter
1	90–100	A
2	80–89	B
3	70–79	C
4	60–69	D
5	0–59	F
6	< 0 or > 100	error

Boundaries to test (both sides):
59/60, 69/70, 79/80, 89/90, plus 0, 100, -1, 101.

Why bins? 101 possible scores, about 14 tests. Same bin, same behavior.

Built live in Part 4, prompt **04**.

Piece 1b: "same bin" is an equivalence

PRINCIPLE: REFLEXIVE / SYMMETRIC / TRANSITIVE TESTS

- same_bin(a, b) := letter_grade(a) = letter_grade(b).
- **Reflexive**: every score is in its own bin. **Symmetric**: if a is with b then b is with a. **Transitive**: a with b, b with c, then a with c.
- Test with helper functions that return the **witness** if a property fails, else nothing.
- This test should be **green on first run**: it records a fact.

Built in Part 4, prompt **05**.

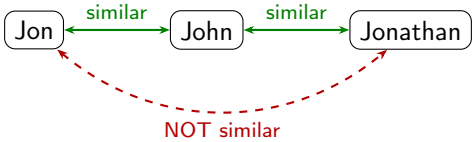
Piece 2: `similar_names`

Pair	Expected	Why
Jon Smith, John Smith	similar	close given names
John Smith, Jonathan Smith	similar	close given names
Jon Smith, Maria Garcia	not similar	different people
Jon Smith, Jon Jones	not similar	family names differ
jon smith, JON SMITH	similar	case does not matter

Rule: same family name, and given-name closeness ≥ 0.5, where closeness $= 1 -$ edit distance/longer length. **Edit distance** = number of single-letter inserts, deletes or changes to turn one word into the other. Prompt **06**.

Piece 3: same_person must be transitive

PRINCIPLE: TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE



Chain Jon John Jonathan, but Jon and Jonathan are **not** linked. Transitivity **fails**, with a concrete witness.

Plan: a property test over the names list that prints the failing triple. Prompt **07**.

Fix 1: a canonical key

PRINCIPLE: EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE

- **Normalize**: rewrite every name to one standard form (nickname table: jon, john $\rightarrow$ jonathan).
- Compare the **keys**: equal keys $\Rightarrow$ same person.
- Equal keys are reflexive, symmetric, transitive **by construction**.
- **Tradeoff**: we only recognize nicknames in our table.
- The old failing test stays recorded as the witness we fixed.

Prompt **08**.

Fix 2: group by chains (the alternative)

PRINCIPLE: EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE

- No key? Link everything connected by *any chain* of matches: **union-find**, also called connected components.
- Jon, John, Jonathan land in one group; the result is a true equivalence.
- **Tradeoff**: it can join two people who never matched directly, if a chain runs between them.

Prompt **09**.

Piece 4: is_duplicate must be symmetric

PRINCIPLE: REFLEXIVE AND SYMMETRIC TESTS

Record A email	Record B email	Bin	Expected
same	same	both present, equal	duplicate
x	y	both present, differ	not duplicate
x	missing	one missing	**not** duplicate (both directions)
missing	missing	both missing	duplicate

Draft rule: a missing email matches anything from *one* side only: is_duplicate(a,b) $\neq$ is_duplicate(b,a). The symmetry test prints the pair. Prompt **10**.

Piece 5: ordering must be antisymmetric

PRINCIPLE: ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE

Bin	Example	Expected
different family names	Garcia vs Smith	Garcia first
same family, different given	Jon Smith vs John Smith	by given name
same name, different id	two "Jon Smith"	by id (no ties)

A comparator returning 0 for *different* people lets the result depend on input order: antisymmetry fails. Fix: add a unique tie-breaker (family, given, id). Prompt **11**.

Piece 6: merge_duplicates

Bin	Input	Expected
empty list (boundary)	[]	[]
no duplicates	distinct people	one merged record each
one group	Jon, John, Jonathan (same email)	one record, ids merged
mixed	some duplicates, some not	only true duplicates merge
shuffled input	same data, any order	same output

The groups of records are the equivalence classes. Prompt **12**.

Piece 7: depends-on order

- A tool reads our own source, finds which function calls which, and returns an order with **leaves first**.
- Test plan: the graph for a small sample module matches a written expected list; the order never puts a function before something it needs.

Prompt **13**.

PRINCIPLE: ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE



If `parse` needs `validate` and `validate` needs `parse`, there is **no leaf to test first**. The order is not antisymmetric. The code should say so: `dependency cycle: parse -> validate -> parse`. Prompt **14**.

Part 4: Build it live

Paste, predict, run, explain.

One new idea per step. Every step ends with a pass or a fail the whole room can see.

- Each slide: the badge (step, red or green), the prompt, one prediction question, what you should see, the recovery.
- If Claude drifts: paste the prompt again, or use the recovery line and move on.
- Reference solution: tags `tdd-step-01` to `tdd-step-19` in the repo.

Step 1: bins and boundaries, red

PRINCIPLE: EQUIVALENCE PARTITIONING, BOUNDARIES

STEP 01 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/04_letter_grade_bins.md

```
Test first. Create merger/__init__.py (empty), a stub merger/grades.py whose
letter_grade(score) just raises NotImplementedError, and tests/test_grades.py.
Spec: 90-100 A, 80-89 B, 70-79 C, 60-69 D, 0-59 F; scores outside 0-100 raise ValueError.
Tests: one parametrized test covering each bin and both sides of every boundary
(59/60, 69/70, 79/80, 89/90) plus 0 and 100, and a test that -1 and 101 raise ValueError.
Run pytest and show me. Every test must be red. Do not implement letter_grade. Stop.
```

1 PREDICT (30 s): All tests red. Which error message?

What you should see: all tests FAILED with `NotImplementedError`.

Recovery: `tools/recover.sh 01`

Common mistake: Letting Claude implement `letter_grade` now. Say: not yet, red first.

Step 2: bins and boundaries, green

PRINCIPLE: EQUIVALENCE PARTITIONING, BOUNDARIES

STEP 02 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/04_letter_grade_bins.md

```
Now make those tests pass with the simplest correct letter_grade. Do not edit the tests.
Run pytest and show me it is all green. Then print the bins: for each letter, the lowest
and highest score 0..100 that gets it. Stop.
```

1 PREDICT (30 s): Will every test pass at once? What bins will it print?

What you should see: all passed; bins A 90–100, B 80–89, C 70–79, D 60–69, F 0–59.

Recovery: `tools/recover.sh 02`

Common mistake: Editing the tests to make them pass. We change the code, not the test.

Step 3: the bins are an equivalence

PRINCIPLE: REFLEXIVE / SYMMETRIC / TRANSITIVE TESTS

STEP 03 • GREEN (green from the start)

1 PREDICT → **2 RUN** → **3 EXPLAIN**

PROMPT FOR CLAUDE

prompts/05_same_bin_equivalence.md

```
Add tests/relations.py with four helpers, each taking (rel, items) where rel(a, b) -> bool:
reflexive_failure, symmetric_failure, antisymmetric_failure, transitive_failure.
Each returns None if the property holds, else the first witness tuple, trying items in order
with itertools.product: (a,) / (a, b) / (a, b) with a != b / (a, b, c).
Then tests/test_grade_relation.py: same_bin(a, b) is letter_grade(a) == letter_grade(b).
Over scores range(0, 101, 3) plus 59, 60, 69, 70, 79, 80, 89, 90, 100, assert it is
reflexive, symmetric and transitive using the helpers; also assert scores 0..100 land in
exactly five distinct letters. Run pytest and show me. These should pass immediately.
```

1 PREDICT (30 s): Red or green? Does a test that never fails count?

What you should see: all green on the first run. Say aloud: "This test passed at once. That is allowed here: it records a fact."

Recovery: `tools/recover.sh 03`

Common mistake: Thinking green-first is cheating. Here it only records a known fact and builds reusable checkers.

Step 4: the fuzzy name rule, red

PRINCIPLE: EQUIVALENCE PARTITIONING

STEP 04 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/06_fuzzy_name_match.md

```
Test first. Create a stub merger/names.py with similar_names(a, b) raising NotImplementedError,
and tests/test_similarity.py. Names are "Given Family". Spec: two names are similar when the
family names are equal (ignoring case) and the given names' closeness is at least 0.5, where
closeness = 1 - edit_distance(given_a, given_b) / max(len of the two given names), lowercase.
Tests: "Jon Smith" ~ "John Smith"; "John Smith" ~ "Jonathan Smith";
"Jon Smith" is NOT similar to "Maria Garcia"; NOT similar to "Jon Jones"; case does not matter
("jon smith" ~ "JON SMITH"). Run pytest, show it red, stop.
```

1 PREDICT (30 s): Red or green? What error will we see?

What you should see: tests FAILED with `NotImplementedError` (stub).

Recovery: `tools/recover.sh 04`

Common mistake: Writing the tests after the code, or editing expected values to fit.

Step 5: the fuzzy name rule, green

STEP 05 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/06_fuzzy_name_match.md

```
Implement similar_names. In merger/names.py put: a helper _split(full_name) returning the
lowercase (given, family) pair; edit_distance(a, b), the usual insert/delete/substitute count;
THRESHOLD = 0.5; and similar_names. Do not edit the tests. Run pytest: all green. Then print
similar_names for every pair among Jon Smith, John Smith, Jonathan Smith. Stop.
```

1 PREDICT (30 s): Will Jon match Jonathan?

What you should see: Jon∼John `True`, John∼Jonathan `True`, **Jon∼Jonathan** `False`. Point at the last line.

Recovery: `tools/recover.sh 05`

Common mistake: Not noticing the False. Stop and point at it; it is the setup for the next step.

Step 6: the transitivity test, red

PRINCIPLE: TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE

STEP 06 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/07_transitivity_counterexample.md

```
Add tests/fixtures.py with SMITH_NAMES = ["Jon Smith", "John Smith", "Jonathan Smith",
"Maya Smith", "Maria Garcia"]. In merger/names.py add: same_person = similar_names
(our first-draft rule for "same person"). Write tests/test_transitivity.py: same_person must
be reflexive, symmetric and transitive over SMITH_NAMES, using the helpers in
tests/relations.py (import them with: from relations import ...). The transitive test must
fail with an assert message (assert witness is None, f"...") "'A' ~ 'B' and 'B' ~ 'C' but 'A' !~ 'C'"
   filled in from the witness.
Run pytest and show me the failure. Do not fix anything. Stop.
```

1 PREDICT (30 s): Will the transitive test pass? Which pair or triple breaks it?

What you should see: exactly one failure, the transitive test, saying `'Jon Smith' ~ 'John Smith'` and `'John Smith' ~ 'Jonathan Smith'` but `'Jon Smith' !~ 'Jonathan Smith'`. The other two pass.

Recovery: `tools/recover.sh 06`

Common mistake: Fixing it right away. First read the witness aloud. A good red test tells you exactly what broke.

Step 7: fix with a canonical key, green

PRINCIPLE: EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE

STEP 07 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/08_canonical_key_fix.md

```
Fix it without touching the three property tests. In merger/names.py add
NICKNAMES = {"jon": "jonathan", "john": "jonathan", "johnny": "jonathan", "jonny": "jonathan"},
canonical_key(full_name) = the nickname-normalized lowercase given name + " " + lowercase
family name, and redefine same_person(a, b) as canonical_key(a) == canonical_key(b).
Keep similar_names unchanged. Add a test that keeps the old counterexample on record:
transitive_failure(similar_names, SMITH_NAMES) == ("Jon Smith", "John Smith", "Jonathan Smith").
Run pytest: all green. Then print the canonical key of every name in SMITH_NAMES. Stop.
```

1 PREDICT (30 s): Will the old failing test pass now, and why?

What you should see: all green; keys `jonathan smith` (three times), `maya smith`, `maria garcia`.

Recovery: `tools/recover.sh 07`

Common mistake: Deleting the old failing test. Keep it: it now passes.

Step 8: chains (union-find), red

PRINCIPLE: EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE

STEP 08 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/09_chain_grouping.md

```
Test first. Stub merger/grouping.py with group_by_chain(items, rel) raising NotImplementedError.
It returns a list of groups (each a list, first-seen order): items are in the same group when
linked by any chain of rel matches. Write tests/test_grouping.py:
1) group_by_chain(SMITH_NAMES, similar_names) == [["Jon Smith","John Smith","Jonathan Smith"],
   ["Maya Smith"], ["Maria Garcia"]];
2) "same group" is reflexive, symmetric and transitive over SMITH_NAMES (use the helpers);
3) tradeoff: Jon Smith and Jonathan Smith are NOT similar_names, yet share a group.
Run pytest, show it red, stop.
```

1 PREDICT (30 s): Red or green? What would a stub return?

What you should see: tests fail (grouping not written yet).

Recovery: `tools/recover.sh 08`

Common mistake: Mixing up the two fixes. This one needs no key; it links everything connected by a chain.

Step 9: chains (union-find), green

PRINCIPLE: EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE

STEP 09 • GREEN

1 PREDICT → **2 RUN** → **3 EXPLAIN**

PROMPT FOR CLAUDE

prompts/09_chain_grouping.md

```
Implement group_by_chain with union-find (connected components), no other dependencies.
Do not edit the tests. Run pytest: all green. Then print the groups for SMITH_NAMES. Stop.
```

1 PREDICT (30 s): How many groups from the five Smith and Garcia names?

What you should see: three groups: the three Smiths together, `Maya Smith`, `Maria Garcia`.

Recovery: `tools/recover.sh 09`

Common mistake: Forgetting the tradeoff: chains can join people who never matched directly.

Step 10: one-sided “equals”, red

PRINCIPLE: REFLEXIVE AND SYMMETRIC TESTS, WITNESS

STEP 10 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/10_symmetric_equals.md

```
Test first. Add merger/records.py: a frozen dataclass Record(id: int, name: str,
email: Optional[str] = None) and the first-draft is_duplicate(a, b) =
same_person(a.name, b.name) and (a.email is None or a.email == b.email).
Add to tests/fixtures.py: RECORDS = [Record(1,"Jon Smith"), Record(2,"John Smith","js@example.edu"),
Record(3,"Jonathan Smith","js@example.edu"), Record(4,"Maya Smith","maya@example.edu"),
Record(5,"Maria Garcia","mg@example.edu")]. Write tests/test_record_equals.py: is_duplicate must
be reflexive and symmetric over RECORDS (use the helpers); the symmetric failure message names both
records. Run pytest. Exactly the symmetric test should fail. Show me and stop.
```

1 PREDICT (30 s): Which pair will the symmetric test print?

What you should see: the symmetric test FAILS and prints a concrete pair of records (one with an email, one without).

Recovery: `tools/recover.sh 10`

Common mistake: Testing only is_duplicate(a, b). The test must also try (b, a).

STEP 11 • GREEN

1 PREDICT → **2 RUN** → **3 EXPLAIN**

PROMPT FOR CLAUDE

prompts/10_symmetric_equals.md

```
Fix is_duplicate so swapping a and b can never change the answer: same_person on the names AND
a.email == b.email. Do not edit the tests. Run pytest: all green. Then print is_duplicate for
Record 1 vs Record 2 and Record 2 vs Record 1. Stop.
```

1 PREDICT (30 s): What will each direction print now?

What you should see: both directions print `False` (a missing email is no longer a wildcard).

Recovery: `tools/recover.sh 11`

Common mistake: Making the rule looser to "fix" the failure. Look at the witness first.

Step 12: sort ties, red

PRINCIPLE: ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE

STEP 12 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/11_antisymmetric_sort.md

```
Test first. Add merger/ordering.py with the first-draft compare(a, b) for Records: compare the
family name only (last word of name, lowercase), returning -1, 0 or 1.
Write tests/test_ordering.py using RECORDS and the helpers, with at_or_before(a, b) meaning
compare(a, b) <= 0: (1) at_or_before is antisymmetric (message names the two records);
(2) it is transitive; (3) sorting RECORDS with functools.cmp_to_key(compare) gives the SAME list
for every permutation of RECORDS (itertools.permutations). Run pytest. Expect the antisymmetric
and the shuffle tests to fail. Show me and stop.
```

1 PREDICT (30 s): Which two names will the witness show?

What you should see: the antisymmetric test FAILS; the witness is `Jon Smith` and `John Smith` (same family name).

Recovery: `tools/recover.sh 12`

Common mistake: Thinking a tie is harmless. Two different people comparing equal makes the sort depend on input order.

Step 13: unique sort key, green

PRINCIPLE: ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE

STEP 13 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/11_antisymmetric_sort.md

```
Fix compare with a unique key: (family name lowercase, given name lowercase, id), so that
compare returns 0 only for the very same record. Put it in a function sort_key(record).
Do not edit the tests. Run pytest: all green. Then print RECORDS sorted. Stop.
```

1 PREDICT (30 s): What order will the names come out in?

What you should see: all pass; sorted order Garcia first, then the Smiths by given name.

Recovery: `tools/recover.sh 13`

Common mistake: Sorting by name only. The key must be unique: family, given, id.

Step 14: merge end to end, red

PRINCIPLE: EQUIVALENCE PARTITIONING

STEP 14 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/12_merge_pipeline.md

```
Test first. Stub merger/pipeline.py with merge_duplicates(records) raising NotImplementedError.
It returns Merged(ids, name, email) tuples (a namedtuple), one per group of duplicates, sorted
by ids; ids is the sorted tuple of the members' ids; name and email come from the lowest-id member.
Write tests/test_pipeline.py with RECORDS: (1) the result as (ids, name) pairs is
[((1,),"Jon Smith"), ((2,3),"John Smith"), ((4,),"Maya Smith"), ((5,),"Maria Garcia")];
(2) every input id appears in exactly one merged record; (3) merge_duplicates(RECORDS) equals
merge_duplicates(list(reversed(RECORDS))). Run pytest, show it red, stop.
```

1 PREDICT (30 s): Red or green? What will fail?

What you should see: the end-to-end merge test fails (not written yet).

Recovery: `tools/recover.sh 14`

Common mistake: Testing the whole thing only. The pieces were tested first, which is why a failure here points at the merge itself.

Step 15: merge end to end, green

PRINCIPLE: EQUIVALENCE PARTITIONING

STEP 15 • GREEN

1 PREDICT → **2 RUN** → **3 EXPLAIN**

PROMPT FOR CLAUDE

prompts/12_merge_pipeline.md

```
Implement it in merger/pipeline.py with exactly these module-level functions:
duplicate_key(record) = (canonical_key(record.name), record.email); group_records(records)
groups by duplicate_key calling duplicate_key; merge_group(group); and merge_duplicates calling
group_records and merge_group. Do not edit the tests. Run pytest: all green. Then print
merge_duplicates(RECORDS), one line each. Stop.
```

1 PREDICT (30 s): How many merged records will it print?

What you should see: four lines; Jon alone (no email), John and Jonathan merged as ids (2, 3).

Recovery: `tools/recover.sh 15`

Common mistake: Counting four people. Four lines, because Jon stays separate and John and Jonathan merge.

Step 16: depends-on order, red

PRINCIPLE: PARTIAL-ORDER TEST ORDERING

STEP 16 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/13_dependency_order.md

```
Test first. Stub merger/deps.py: build_graph(*modules) and leaves_first(graph), both raising
NotImplementedError. build_graph maps each top-level function name in the given modules to the
set of OTHER top-level functions (from those modules) it calls by plain name.
Write tests/test_dependencies.py for modules merger.names, merger.records, merger.pipeline:
(1) graph["merge_duplicates"] == {"group_records","merge_group"}, graph["group_records"] ==
{"duplicate_key"}, graph["duplicate_key"] == {"canonical_key"}, graph["canonical_key"] ==
{"_split"}, graph["_split"] == set(); (2) leaves_first(graph) lists every function after
everything it depends on, and its first item depends on nothing; (3) the transitive closure of
"depends on" has no function depending on itself and is antisymmetric and transitive (helpers).
Run pytest, show it red, stop.
```

1 PREDICT (30 s): Red or green? What will the test compare?

What you should see: the dependency tests fail (graph tool not written yet).

Recovery: `tools/recover.sh 16`

Common mistake: Writing the graph by hand. We let the tool read our own code.

Step 17: depends-on order, green

STEP 17 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/13_dependency_order.md

```
Implement build_graph with the ast module (parse the source with inspect.getsource; look at
ast.Call nodes whose func is a Name) and leaves_first with a depth-first walk that visits
dependencies first. Do not edit the tests. Run pytest: all green. Then print the graph and the
leaves_first order, one function per line. Stop.
```

1 PREDICT (30 s): Which function comes first in the order, and which last?

What you should see: graph lines as in the test; order starts with `_split`, `canonical_key`, ... and ends with `merge_duplicates` (others interleaved).

Recovery: `tools/recover.sh 17`

Common mistake: Expecting one right answer. Several orders are valid; only the rule matters.

Step 18: cycle detection, red

PRINCIPLE: ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE

STEP 18 • RED

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/14_cycle_design_smell.md

```
Test first. In merger/deps.py add class CycleError(Exception). Write tests/test_cycles.py with
LOOP = {"parse": {"validate"}, "validate": {"parse"}, "report": {"validate"}}:
(1) leaves_first(LOOP) raises CycleError whose message contains "parse" and "validate";
(2) using the transitive-closure code from test_dependencies.py (import it:
from test_dependencies import closure; if there is no function called closure, extract it there
first without changing behaviour), antisymmetric_failure on LOOP's closure is ("parse","validate").
Run pytest. The cycle test should fail (the second test may already pass: say so). Stop.
```

1 PREDICT (30 s): What should a cycle report look like?

What you should see: the cycle test fails (not detected yet).

Recovery: `tools/recover.sh 18`

Common mistake: Dismissing cycles as rare. They are a design smell: no leaf to test first.

STEP 19 • GREEN

1 PREDICT → 2 RUN → 3 EXPLAIN

PROMPT FOR CLAUDE

prompts/14_cycle_design_smell.md

```
Make leaves_first detect cycles: track the current path and raise CycleError with a message like
"dependency cycle: parse -> validate -> parse". Do not edit the tests. Run pytest: all green.
Then print the message for LOOP. Stop.
```

1 PREDICT (30 s): What message will the program print?

What you should see: `dependency cycle: parse -> validate -> parse`

Recovery: `tools/recover.sh 19`

Common mistake: Treating the cycle as a test problem. It is a design problem; the fix is a different structure.

You just saw the whole process

Idea $\rightarrow$ proposal $\rightarrow$ pieces $\rightarrow$ plan $\rightarrow$ tests first $\rightarrow$ working code.

- Every concept earned its place by catching a real bug: transitive (Jon, John, Jonathan), symmetric (missing email), antisymmetric (tie), partial order (test order, cycles).
- **Thursday: you repeat this** in pairs on a *different* problem, with your own proposal, plan, tests and witnesses. You may use Claude; you write the witnesses.

Student workbook: `week-11/tdd-walkthrough/student/`

Index 1 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
10	A relation is a set of pairs	WITNESS / COUNTEREXAMPLE	
11	See it: pairs, matrix, digraph	WITNESS / COUNTEREXAMPLE	01a
13	Four questions, four witnesses	WITNESS / COUNTEREXAMPLE	02a
14	Two big kinds of relation	EQUIVALENCE VS PARTIAL ORDER	
15	Sort relations into kinds	EQUIVALENCE VS PARTIAL ORDER	03a
21	Rules of the game	PARTIAL-ORDER TEST ORDERING	00a
23	Depends on: the plan as arrows	PARTIAL-ORDER TEST ORDERING	
24	Test order: leaves first	PARTIAL-ORDER TEST ORDERING	
25	Piece 1: letter_grade	EQUIVALENCE PARTITIONING, BOUNDARIES	

Index 2 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
26	Piece 1b: “same bin” is an equivalence	REFLEXIVE / SYMMETRIC / TRANSITIVE TESTS	
27	Piece 2: similar_names	EQUIVALENCE PARTITIONING	
28	Piece 3: same_person must be transitive	TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE	
29	Fix 1: a canonical key	EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE	
30	Fix 2: group by chains (the alternative)	EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE	
31	Piece 4: is_duplicate must be symmetric	REFLEXIVE AND SYMMETRIC TESTS	
32	Piece 5: ordering must be antisymmetric	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	
33	Piece 6: merge_duplicates	EQUIVALENCE PARTITIONING	
34	Piece 7: depends-on order	PARTIAL-ORDER TEST ORDERING	

Index 3 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
35	Piece 8: cycles are a design smell	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	
37	Step 1: bins and boundaries, red	EQUIVALENCE PARTITIONING, BOUNDARIES	04a
38	Step 2: bins and boundaries, green	EQUIVALENCE PARTITIONING, BOUNDARIES	04b
39	Step 3: the bins are an equivalence	REFLEXIVE / SYMMETRIC / TRANSITIVE TESTS	05a
40	Step 4: the fuzzy name rule, red	EQUIVALENCE PARTITIONING	06a
41	Step 5: the fuzzy name rule, green	EQUIVALENCE PARTITIONING	06b
42	Step 6: the transitivity test, red	TRANSITIVE TEST, WITNESS / COUNTEREXAMPLE	07a
43	Step 7: fix with a canonical key, green	EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE	08a
44	Step 8: chains (union-find), red	EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE	09a

Index 4 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
45	Step 9: chains (union-find), green	EQUIVALENCE PARTITIONING, WITNESS / COUNTEREXAMPLE	09b
46	Step 10: one-sided “equals”, red	REFLEXIVE AND SYMMETRIC TESTS, WITNESS	10a
47	Step 11: symmetric equals, green	REFLEXIVE AND SYMMETRIC TESTS, WITNESS	10b
48	Step 12: sort ties, red	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	11a
49	Step 13: unique sort key, green	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	11b
50	Step 14: merge end to end, red	EQUIVALENCE PARTITIONING	12a
51	Step 15: merge end to end, green	EQUIVALENCE PARTITIONING	12b
52	Step 16: depends-on order, red	PARTIAL-ORDER TEST ORDERING	13a
53	Step 17: depends-on order, green	PARTIAL-ORDER TEST ORDERING	13b

Index 5 of 5: principle, slide, prompt

Slide	Title	Principle	Prompt
54	Step 18: cycle detection, red	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	14a
55	Step 19: cycle detection, green	ANTISYMMETRIC TEST, WITNESS / COUNTEREXAMPLE	14b